Web Crawling - Full Explanation

1. What is Web Crawling?

Web Crawling (also called web scraping or spidering) is the process of automatically browsing the web to collect data from websites. A web crawler (or spider) is a program that:

1. Starts from one or more URLs (called seed URLs).
2. Downloads their HTML content.
3. Extracts useful information (like text or links).
4. Follows the links to other pages and repeats the process.

2. Why Crawling is Used

Use Case	Description
Search engines	Google, Bing, etc. crawl websites to index pages for searching.
Data analysis	Collect product data, news, or research papers for analysis.
AI training	Collect text datasets to train NLP or machine learning models.
Price comparison	Gather product prices from e-commerce websites.
Academic research	Retrieve articles, blogs, or social media posts.

3. How a Crawler Works

1. Start with a URL → Example: https://en.wikipedia.org/wiki/Information_retrieval

2. Download the page (using an HTTP library like requests).
3. Parse the content using an HTML parser (like BeautifulSoup).
4. Extract information such as visible text and links (<a href="...">).
5. Follow links and repeat the process.
6. Store results (in a file, database, etc.).

4. The Role of robots.txt

Before crawling a website, crawlers must check a special file called robots.txt, located in the site's root directory.
Example: https://www.wikipedia.org/robots.txt

It contains rules like:
User-agent: *
Disallow: /private/
Allow: /public/

User-agent = the crawler's name.
Disallow = paths the crawler must not visit.
Allow = paths that are okay.

Your crawler should respect these rules to avoid legal or ethical issues.

5. Python Tools Used

requests → Sends HTTP requests to get web pages.
BeautifulSoup → Parses HTML and extracts data (from tags, text, etc.).
robotparser → Reads and interprets robots.txt files.
urllib.parse → Handles URL parsing and joining (e.g., absolute vs relative links).

6. Step-by-Step Code Explanation

The provided Python code implements a simple web crawler that:
1. Checks robots.txt to ensure access permission.
2. Downloads pages using requests.
3. Parses text and links using BeautifulSoup.
4. Stores data (text snippet + links).
5. Recursively crawls linked pages up to a defined depth.

7. Summary

Step	Action	Tool
1	Start from a URL	—
2	Check robots.txt	robotparser
3	Fetch the HTML page	requests
4	Parse text and links	BeautifulSoup
5	Store data	List or file
6	Follow links recursively	Function _crawl()

8. Important Notes

- Always respect robots.txt.
- Avoid overloading servers (add delays between requests).
- Use a custom user-agent string like "MyCrawler/1.0".
- Never crawl sensitive or private data.